



Log Parsing Neden Zor Bir Problem?



Gün 1: Loglarla Tanışma

Serbest metin formatındaki karmaşık loglardan, yapılandırılmış ve analiz edilebilir veriye giden yol.

Neden Log Parsing Zor Bir Problem?

Milyonlarca satırlık sistem loglarını yapılandırmak görüldüğünden çok daha zordur çünkü:

- Zaman damgası, değişken içerikler (ID'ler, IP'ler) ve sabit şablon metinleri birbirine girmiş durumdadır.
- Sistemin farklı bileşenleri (DataNode, NameNode vb.) tamamen farklı formatlarda log üretir.
- Gözle ayırt etmesi kolay olan desenleri algoritmik olarak genellemek, özellikle RegEx ile yazıldığında, yeni log türleri geldiğinde anında kırılır.
- Saniyede binlerce log üreten sistemlerde bu işlemi gecikmesiz (online) yapmak devasa bir performans gerektirir.



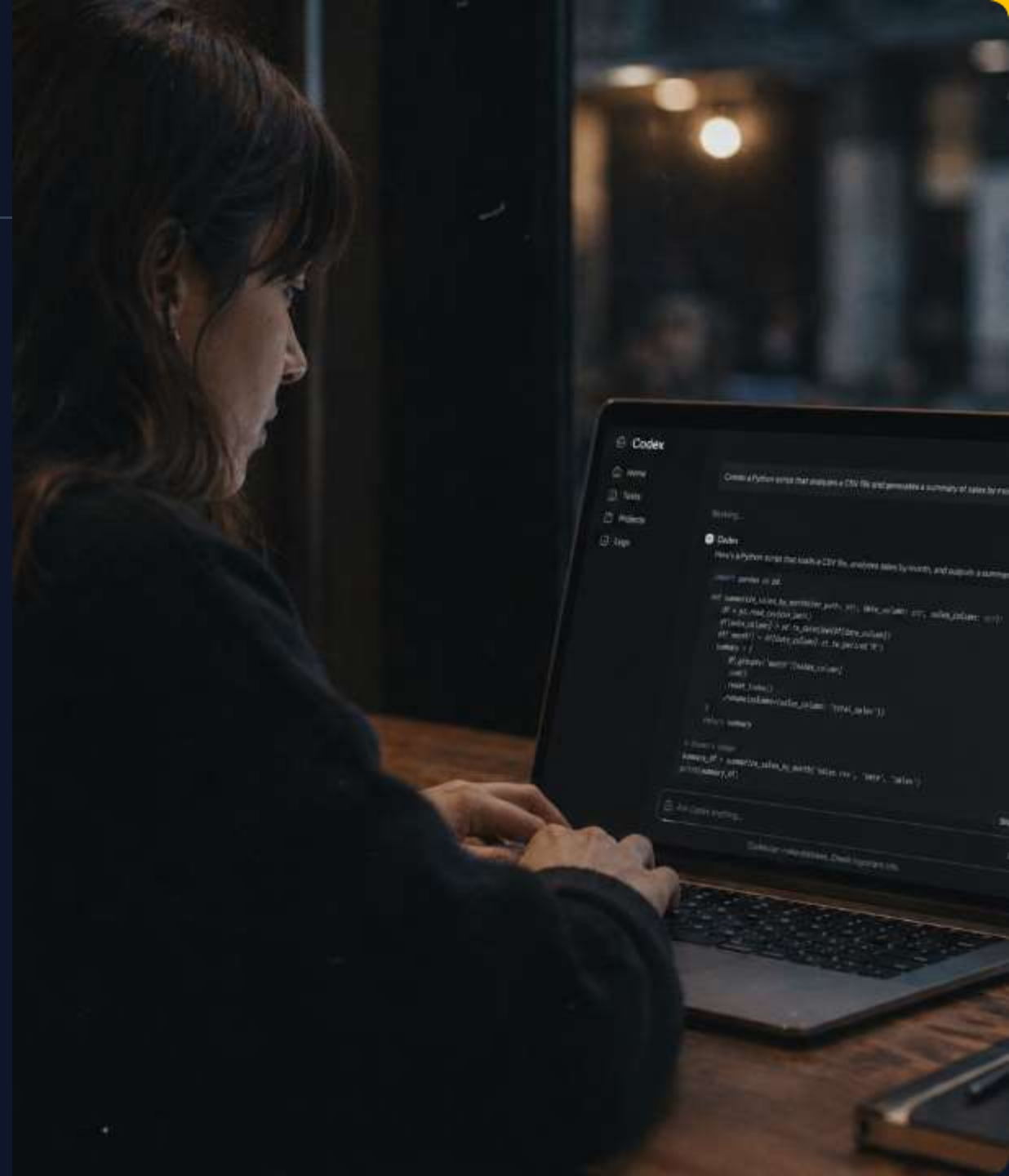
HDFS_2k.log Manuel İnceleme

Kendi Gözümüzle Ayırt Etme Deneyimi

LogHub'dan indirdiğimiz HDFS_2k.log dosyasından hiçbir ayırıştırıcı kullanmadan rastgele satırları okuduk.

"Receiving block" veya "src:." gibi sistemin her defasında aynı bastığı sabit kısımlar ile, "blk_-16089996879" veya "10.250.19.102" gibi her olayda değişen kısımları manuel olarak ayırdık.

Bu "İsinma türü", kural tabanlı bir ayırıştırıcı yazmanın sınırlarını ve Drain3 gibi otonom algoritmalara olan mutlak ihtiyacı net bir şekilde hissettirdi.





Gün 2: Drain3 ile Otomasyon

Log ağacı mimarisi ile sabit token'ları değişkenlerden
(parametrelerden) otonom olarak ayırma.



Drain3'ün Çalışma Mantığı



1. Uzunluk Ayrımı

Algoritma öncelikle log mesajını kelime (token) sayısına göre sınıflandırır. Farklı uzunluktaki loglar farklı dallara ayrılarak arama uzayı hızla daraltılır.



2. Token Ağacı

Sabit kelimeler ağaç yapısında düğümler (node) olarak tutulurken, IP veya ID gibi değişken kısımlar <*> (wildcard) işareti ile maskelenir.



3. Benzerlik Eşiği

Yeni gelen bir log, mevcut düğümlerle karşılaştırılır. Belirlenen bir benzerlik oranını aşarsa o şablona dahil edilir, aşmazsa yeni bir şablon dalı oluşturulur.



İlk Template Çıkarımı Sonucu

14

Benzersiz Template

Büyük Verinin Sıkıştırılması

Drain3, 2.000 satırlık ham HDFS log kümesini saniyeler içinde işleyerek tüm o karmaşık yapıyı sadece **14 farklı olay şablonuna (template)** indirgemeyi başardı.

Bu, değişken verilerin maskelenerek arka planda yatan ana sistem davranışlarının nasıl başarılı bir şekilde soyutlandığının en büyük kanıtıdır.



En Sık Görülen 5 Şablon (Top 5)

Çıkarılan Log Şablonu (Template)	Görülme Sıklığı
Receiving block <*> src: /<*> dest: /<*>	845
BLOCK* NameSystem.allocateBlock: <*> <*>	452
PacketResponder <*> for block <*> terminating	231
Received block <*> of size <*> from /<*>	185
BLOCK* NameSystem.addStoredBlock: blockMap updated: <*> is added to <*> size <*>	110



Gün 3: Başarıyı Ölçme

Template dağılımını analiz etme ve anomali tespiti için tohumları atma.



Template Dağılım Analizi



Logların ezici bir çoğunluğu birkaç rutin işlem şablonuna (blok alma, tahsis etme) düşmektedir. Dağılımın kuyruğunda yer alan şablonlar, sistemdeki istisnai durumları temsil eder.



Parse Başarısını Nasıl Değerlendiriyoruz?



Hassasiyet ve Genelleme Dengesi: Mükemmel bir parser, her bir log için ayrı template üretmek (under-parsing) ile her şeyi tek template'e sıkıştırmak (over-parsing) arasındaki o ince dengeyi bulmalıdır. 14 template bu dengenin HDFS verisindeki optimal noktasıdır.



Online Eşleşme Hızı: Drain3'ün derinliği kısıtlanmış ağaç yapısı, yeni gelen bir log satırının hangi şablona ait olduğunu mikrosaniyeler içinde bularak gerçek zamanlı işlem yapmaya olanak tanır.



Beklenmedik Loglar (Anomali Tohumları): Dağılımda gördüğümüz sadece 1-2 kez eşleşen o nadir template'ler; aslında hata mesajları, timeout'lar veya donanımsal arızaların ilk işaret fişekleridir. *Gelecek haftaki görevimizin temeli budur.*



Soru & Cevap

Gelecek hafta başlayacağımız "[Anomali Tespiti](#)" aşamasına geçmeden önce sorularınızı alabilirim.

Ahmet Ayduğan

Monitoring & DevOps